

Scheduler Activations

Definition

Scheduler activations are a technique designed to combine the good performance and flexibility of user-level threads with the useful blocking behavior of kernel threads.

1. Why Scheduler Activations Were Proposed

Main Motivation

Kernel threads solve important problems such as blocking system calls and page faults, but they are slower. User-level threads are faster, but they block badly when the kernel becomes involved. Scheduler activations try to get the advantages of both.

Main Goal

The goal is to make user-space threads behave as if they had the functionality of kernel threads, while still keeping much of the performance and flexibility of user-level thread packages.

2. What the Model Tries to Achieve

Desired Properties

With scheduler activations:

- user threads should not need special nonblocking system calls,
- user threads should not have to test in advance whether a system call is safe,
- if one thread blocks on a system call or page fault, another ready thread in the same process should still be able to run.

Efficiency Principle

The system tries to avoid unnecessary transitions between user space and kernel space. If a thread blocks only because of user-level synchronization, there is no reason to involve the kernel.

3. Virtual Processors

Core Mechanism

The kernel assigns each process some number of **virtual processors**. The user-space run-time system then decides which user threads run on those virtual processors.

Initial Allocation

A process usually starts with one virtual processor, but:

- it can request more virtual processors,
- it can return processors it no longer needs,
- and the kernel can take processors back if another process needs them more.

Multiprocessor Case

On a multiprocessor system, these virtual processors may correspond directly to real CPUs.

4. Upcalls

Upcall

When the kernel learns that a thread has blocked or become runnable again, it notifies the process's run-time system by activating it at a known entry point. This notification mechanism is called an **upcall**.

What the Kernel Passes

When a blocking event occurs, the kernel informs the run-time system by passing:

- the identity of the affected thread,
- a description of the event that occurred.

UNIX Analogy

This is somewhat like a UNIX signal, except it is specifically used to notify the thread run-time system about thread-related kernel events.

5. What Happens When a Thread Blocks

Blocking Case

Suppose a thread blocks because of a blocking system call or a page fault.

Kernel Action

The kernel does not simply leave the whole process stuck. Instead, it performs an upcall to the process's run-time system.

Run-Time System Action

After receiving the upcall, the run-time system can:

- mark the current thread as blocked,
- choose another ready thread,
- load that thread's state,
- and resume execution.

6. What Happens When the Blocked Thread Becomes Ready Again

Unblocking Case

Later, the kernel may discover that the blocked thread can run again. For example:

- input becomes available,
- a pipe receives data,
- or a missing page is loaded from disk.

Second Upcall

The kernel then makes another upcall to notify the run-time system that the thread is ready again.

Run-Time Choice

The run-time system can then:

- restart that thread immediately, or
- place it on the ready list and schedule it later.

7. Interrupt Handling

Ordinary Interrupt Case

If a hardware interrupt happens while a user thread is running, the CPU switches into kernel mode.

Case 1: Interrupt Not Relevant

If the interrupt is unrelated to the current process, such as another process's I/O completion, then after the interrupt handler finishes, the interrupted thread is resumed as before.

Case 2: Interrupt Relevant

If the interrupt is relevant to the current process, such as the arrival of a page needed by one of its threads, then the kernel does not simply resume the interrupted thread. Instead, it starts the run-time system on that virtual CPU and lets it decide what thread should run next.

Possible Scheduling Choices

After the upcall, the run-time system may choose:

- the interrupted thread,
- the newly ready thread,
- or some other ready thread.

8. Why Scheduler Activations Are Efficient

Key Efficiency Idea

Scheduler activations reduce overhead by involving the kernel only when necessary. Purely user-level synchronization stays in user space, while kernel-relevant blocking events are still reported correctly.

Result

This preserves much of the speed and flexibility of user-level thread scheduling while avoiding the worst blocking problems of pure user-level threads.

9. Main Criticism

Layering Problem

Scheduler activations rely heavily on **upcalls**, and upcalls break the normal layering rule of system design.

Why This Is a Problem

In a normal layered system, layer n provides services to layer $n + 1$, but does not call procedures in layer $n + 1$. Upcalls violate this rule because the kernel calls into the user-level run-time system.

Meaning

This makes the design conceptually powerful, but architecturally controversial.

10. Comparison with Other Models

Feature	User-Level Threads	Kernel Threads	Scheduler Activations
Speed	High	Lower	Tries to stay high
Blocking system call handling	Poor	Good	Good
Need for wrappers	Often yes	No	No
Scheduling flexibility	High	Lower	High
Kernel involvement	Minimal	Heavy	Only when needed

11. Main Conclusion

Takeaway

Scheduler activations try to achieve the best of both worlds: the speed and flexibility of user-level threads plus the correct blocking behavior of kernel threads. Their key idea is that the kernel informs the user-level run-time system through upcalls whenever thread-relevant events occur.

12. Quick Review

Exam-Focused Points

- Scheduler activations were designed to combine user-level and kernel-thread advantages.
- The kernel gives each process some number of virtual processors.
- The user-level run-time system schedules user threads onto those virtual processors.
- When a thread blocks or becomes ready again, the kernel informs the run-time system using an upcall.
- Upcalls let the run-time system choose another thread without losing flexibility.
- Scheduler activations avoid the need for wrappers and special nonblocking system calls.
- They are efficient because unnecessary user-kernel transitions are avoided.
- Their main criticism is that upcalls violate the normal layered-system structure.